PROJECT REPORT

Automated SOAR Playbook: Used Shuffle SOAR to Automatically Isolate Endpoints Based on Malicious Events

Project domain: Security Orchestration, Automation and Response (SOAR)
Platform: Shuffle SOAR
Institution branding: Sandip University logo supplied for this report

Abstract

This project demonstrates an event-driven SOAR workflow using Shuffle. Security events are normalized, evaluated against conservative criteria, and passed to a workflow for enrichment, policy checks, approval, and potential endpoint containment. The included Python utility evaluates a sample event and can notify a Shuffle HTTPS webhook. For safety, the utility itself never isolates endpoints; disruptive actions must be implemented and reviewed within an authenticated Shuffle workflow with endpoint allowlisting, audit logging, human approval where appropriate, and a tested recovery path.

1. Introduction

Security operations teams receive alerts from endpoint detection, SIEM, identity, and network tools. Responding manually to every high-confidence malicious event can delay containment. SOAR platforms coordinate enrichment and response actions through workflows, helping teams apply consistent decisions and retain an audit trail. Automation must be carefully gated because false positives or incorrect endpoint identity can disrupt business operations.

2. Problem Statement

A malicious event may require fast investigation and containment, but a fully automatic isolation action based on a single unverified signal can cause outages. The project addresses the need for a repeatable workflow that evaluates event context, routes candidates to Shuffle, validates endpoint identity and policy, and records outcomes before a containment action is approved.

3. Objectives

- Create a Python utility to parse and evaluate normalized security events.
- Demonstrate optional HTTPS notification to a Shuffle workflow webhook.
- Define playbook gates for event validation, severity, endpoint allowlisting, approval, and rollback.
- Document audit logging and outcome verification.
- Use synthetic events to validate the workflow without impacting production endpoints.

4. Scope and Safety

The included Python script performs triage evaluation and optional webhook notification only. It does not directly isolate endpoints, install agents, or configure an EDR platform. Actual containment must be implemented in a separately reviewed Shuffle workflow using an authorized endpoint integration. Start in a lab with synthetic events and dedicated endpoints. Use least-privilege credentials and do not commit secrets to source control.

5. Proposed Architecture

Stage	Responsibility
Event source	Provides a normalized security alert from a lab SIEM/EDR or synthetic test file.

Stage	Responsibility
Python evaluator	Checks event type, severity, and explicit indicator; records a review decision.
Shuffle webhook	Receives the decision and starts the configured workflow.
Policy and enrichment	Validates schema, deduplicates events, enriches context, checks allowlists, and applies confidence thresholds.
Approval and containment	Requires authorized approval when appropriate, then invokes a configured EDR integration and verifies the result.
Audit and recovery	Records the action and outcome, notifies responders, and provides a tested rollback/reconnect procedure.

6. Methodology

1) Define the authorized scope and identify a dedicated lab endpoint. 2) Prepare a synthetic event payload. 3) Configure Shuffle to accept the webhook and validate the payload schema. 4) Add deduplication, enrichment, endpoint allowlisting, severity/confidence checks, and an approval gate. 5) Configure an authorized EDR/endpoint integration only after testing its permissions. 6) Run the Python utility against the sample event and verify the decision. 7) Test webhook delivery in a non-production workflow. 8) Verify containment and recovery procedures only on approved lab endpoints. 9) Record workflow execution logs and test results.

7. Decision Logic

The sample evaluator marks an event as a candidate when an explicit known-malicious indicator is present, or when severity meets a configurable threshold and the event type is in a small allowlist. The result is a review decision, not a final malware verdict. Missing endpoint identity triggers a review state. The Shuffle workflow must independently validate identity and policy before any disruptive response.

8. Example Execution

```
python automated_soar_playbook.py --input sample_security_event.json
# Optional webhook notification after configuring an HTTPS URL:
python automated_soar_playbook.py --input sample_security_event.json --notify
```

When --notify is used, set SHUFFLE_WEBHOOK_URL in the environment. Use a dedicated test workflow and keep the webhook URL secret.

9. Audit and Metrics

Useful metrics include events received, candidates flagged, approval rate, false-positive rate, webhook failures, containment success rate, time from event to decision, and recovery outcomes. Preserve timestamps, event IDs, workflow run IDs, approver identity, endpoint identity, and action results while minimizing sensitive data in logs.

10. Expected Outcomes

- A repeatable event-to-workflow integration pattern.
- Consistent review decisions based on documented criteria.
- A design for gated containment, approval, audit, and recovery.
- Reduced manual coordination for validated events when implemented and tested with authorized integrations.

11. Limitations

This package is a template, not a deployed SOAR service. It does not automatically configure Shuffle nodes, integrate a specific EDR product, or isolate endpoints. The demonstration rule is intentionally simple and must not be used as the sole basis for production response. A successful webhook HTTP response confirms delivery only, not workflow success. Integration details vary by Shuffle version and endpoint platform.

12. Conclusion

The Automated SOAR Playbook demonstrates how security events can be evaluated and routed to Shuffle for structured response. By separating event triage from the disruptive containment action, the design emphasizes policy checks, endpoint allowlisting, approval, auditability, and recovery. Before claiming a completed deployment, validate

the workflow in an isolated lab and include sanitized screenshots and verified test results.

References and Further Reading

• Official Shuffle SOAR documentation.
• NIST SP 800-61 — Computer Security Incident Handling Guide.
• MITRE ATT&CK; — techniques and defensive context for observed behavior.
• Vendor documentation for the selected EDR/endpoint isolation integration.

Prepared as an educational project template. Implement and test actual endpoint isolation only through an authorized, reviewed workflow in a controlled environment.